

ESP32 Wecker



ESP32 Sonnenaufgangs-Wecker

Ein Sonnenaufgangs-Wecker auf Basis von ESP32 und MicroPython.
Simuliert einen natürlichen Sonnenaufgang mit sanften LED-Farbübergängen und MP3-Tönen — vollständig auf dem Gerät über den Drehgeber konfigurierbar.

MicroPython

1.24.1

ESP32

D1 Mini

license

MIT

Einrichtung: siehe [INSTALLATION.md](#) · **Technisches Design:** siehe [docs/SYSTEM_ARCHITECTURE.md](#)

Funktionen

- **Sonnenaufgang-Simulation** — 10-minütiger LED-Übergang: Dunkelrot → Orange → Gelb → Weiß
- **6 MP3-Wecktöne** — mit 5-Sekunden-Vorschau im Menü und schrittweiser Lautstärkeerhöhung
- **Gerätemenü** — Weckzeit, Ein/Aus, Ton, Uhrzeit und LED-Umschalter direkt am Gerät per Drehgeber einstellen
- **Schlummerfunktion** — setzt nach 5 Minuten am gleichen LED/Lautstärkepunkt fort
- **Nachtlicht** — Schlummer-Pad im Dunkeln berühren für 30 s weißes Licht, dann 10 s Ausblenden
- **Automatische Helligkeit** — Display passt sich dem Umgebungslicht an (EMA-geglättet)
- **Einstellungen bleiben erhalten** — werden im Flash gespeichert und überstehen Stromausfälle
- **Genaue Zeitführung** — DS3231 RTC (mit Batteriepuffer); Zeit wird zwischen Lesevorgängen extrapoliert
- **Stromausfallserkennung** — zeigt bAt? beim Start, wenn die RTC-Pufferbatterie leer ist

Hardware

Komponente	Modell	GPIO
Mikrocontroller	ESP32 D1 Mini	—
Display	TM1637 7-Segment	CLK: 18, DIO: 19
RTC	DS3231	SDA: 21, SCL: 22
MP3-Player	DFPlayer Mini	TX: 17, RX: 16
LED-Streifen	WS2812B (5 LEDs)	DATA: 27
Encoder	KY-040	CLK: 32, DT: 25, SW: 15
Touch Schlummer	ESP32 Touch	GPIO 4
Touch Stopp	ESP32 Touch	GPIO 0
Lichtsensord	Analog	GPIO 36

WS2812B benötigt 5 V — nicht vom 3,3-V-Pin des ESP32 betreiben.

Dateien

src/	Alle Dateien auf den ESP32 hochladen
main.py	Hauptprogramm
config.py	Pin-Belegungen und Konstanten
tm1637.py	Display-Treiber
ds3231.py	RTC-Treiber
dfplayer.py	DFPlayer Mini-Treiber
rotary_encoder.py	Drehgeber-Treiber
tests/	Optionale Hardware-Testskripte
test_display.py	
test_dfplayer.py	
test_encoder.py	
test_leds.py	
test_light_sensor.py	
test_rtc.py	
test_touch.py	
docs/	
SYSTEM_ARCHITECTURE.md	
CHANGELOG.md	
INSTALLATION.md	
LICENSE	
requirements.txt	Host-Tools: esptool, mpremote

Bedienung

Steuerung

Eingabe	Alarm inaktiv	Alarm aktiv
Encoder drehen	Menü navigieren / Wert ändern	Ignoriert
Encoder klicken	Auswählen / Bestätigen / Nächstes Feld	Alarm stoppen
GPIO 4 berühren	Nachtlicht (wenn dunkel)	Schlummer 5 min
GPIO 0 berühren	—	Alarm stoppen

Das Menü kehrt nach **5 Sekunden** Inaktivität zur Zeitanzeige zurück.

Menü

#	Anzeige	Funktion	Bedienung
0	ALRM	Weckzeit einstellen	Klick → Stunden drehen → Klick → Minuten drehen → Klick zum Speichern
1	ON - F	Alarm Ein/Aus	Klick → drehen zum Umschalten (LEDs: grün = AN, rot = AUS) → Klick zum Speichern
2	SOND	Weckton	Klick → drehen zum Durchblättern (spielt 5-s-Vorschau) → Klick zum Speichern
3	CLOC	Uhrzeit stellen	Klick → Stunden drehen → Klick → Minuten drehen → Klick zum Speichern
4	LED	Sonnenaufgang-LEDs Ein/Aus	Klick → drehen zum Umschalten → Klick zum Speichern

Alarmablauf

1. LEDs beginnen bei Dunkelrot; Ton startet leise
2. Über 10 Minuten: LEDs blenden zu Weiß auf; Lautstärke erhöht sich auf

Maximum

3. Nach 10 Min: volle Helligkeit und Lautstärke; Titel wiederholt sich
4. **Schlummer** (GPIO 4 berühren): pausiert für 5 Min, setzt dann am gleichen Punkt fort
5. **Stopp** (Encoder klicken): stoppt sofort
6. Automatischer Stopp nach 30 Minuten

Gespeicherte Einstellungen

Werden in `alarm_settings.json` im ESP32-Flash gespeichert:

Einstellung	Menü	Standard
Weckstunde & -minute	ALRM	07:00
Alarm aktiviert	ON-F	aus
Ton (0–5)	SOND	0 (BIRD)
Sonnenaufgang-LEDs	LED	an

Die Uhrzeit wird vom DS3231 RTC mit eigener CR2032-Batterie gespeichert.

Fehlersuche

Symptom	Lösung
bAt? beim Start	CR2032 am DS3231 ersetzen; Zeit über CLOC einstellen
00:00 beim Start	RTC nicht gestellt — CLOC-Menü verwenden
Uhr nach Stromausfall falsch	RTC-Batterie leer — CR2032 ersetzen
Bleibt bei INIT stehen	USB abziehen und wieder einstecken (I2C-Bus blockiert)
Kein Ton	SD-Karte prüfen: FAT32, Dateien müssen 0001.mp3–0006.mp3 heißen
Nachtlicht löst nicht aus	Raum zu hell — NIGHT_LIGHT_THRESHOLD in config.py verringern
LEDs leuchten nicht	5-V-Versorgung der WS2812B prüfen

Symptom	Lösung
Touch-Schlummer reagiert nicht	test_touch.py ausführen; TOUCH_THRESHOLD_MIN/MAX in config.py anpassen

Konfiguration

Wichtige Konstanten in src/config.py:

Konstante	Standard	Bedeutung
SUNRISE_DURATION	600 s	Aufwachrampe des Alarms
SNOOZE_DURATION	300 s	Schlummerdauer
ALARM_MAX_DURATION	1800 s	Automatischer Stopp nach dieser Zeit
MENU_TIMEOUT	5000 ms	Menü-Timeout bei Inaktivität
NIGHT_LIGHT_ON_DURATION	30 s	Nachtlicht voll an
NIGHT_LIGHT_DIM_DURATION	10 s	Nachtlicht Ausblendzeit
NIGHT_LIGHT_THRESHOLD	80	EMA-Wert, unterhalb dessen Nachtlicht aktiviert
DFPLAYER_VOLUME_MAX	25	Maximale Alarmlautstärke (0–30)

Lizenz

MIT — siehe [LICENSE](#).